A good way of interpreting these statistics is as follows: A high ratio of *pages_sharing* to *pages_shared* indicates good sharing. A high ratio of *pages_unshared* to *pages_sharing* indicates that KSM is working very hard, yet is unable to succeed in sharing pages.

KSM with and without virtualisation

The primary user of KSM is expected to be virtualisation solutions and KVM in particular. However, KSM is designed to be generic. An example (borrowed from the paper *Increasing memory density by using KSM*) is the usage of KSM at CERN and Berkley National Laboratory.

They had jobs that generated data with a lot of similar content. Enabling KSM allowed them to run three jobs in parallel as compared to two earlier, and allowed the jobs to share 750 MB of the 2 GB data size as compared to 250 MB earlier.

In short, KSM is suited for applications beyond virtualisation, specifically if there is a chance of a large amount of content being the same.

KSM API

KSM provides two main helpers to an existing system call [*madvise(2)*] that we have seen earlier. The two key helper functions are passed as hints:

1. MADV_MERGEABLE: This hint specifies a range of addresses that can be marked as shared.
2. MADV_UNMERGEABLE: This hint specifies a range of addresses that can be now marked as unshared. It is used to cancel a hint given earlier.

KSM in Fedora 12

KSM is enabled by default in Fedora 12; the following program shows usage of the API listed above. However, the header files (*sys/mman.h*) do not have the correct version of the header that exposes the values of MADV_MERGEABLE and MADV_UNMERGEABLE. Our program defines them after picking up the correct values from the kernel source.

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

#define MADV_MERGEABLE  12      /* KSM may merge identical pages */
#define MADV_UNMERGEABLE 13     /* KSM may not merge identical pages */
#define NR_PAGES        100     /* Number of pages to allocate */

int main(void)
{
  size_t ps = getpagesize();
  char *addr;
  int ret, i;

  printf("Page size is %d, allocating %lu\n", ps, (unsigned long)ps*NR_PAGES);
  ret = posix_memalign((void **)&addr, ps, ps*NR_PAGES);
  if (ret != 0) {
    fprintf(stderr, "failed to allocated %d pages\n", NR_PAGES);
    exit(1);
  }
  printf("Allocated the pages at addr %p\n", addr);


  for (i = 0; i < NR_PAGES; i++) {
    memset(addr+i*ps, i, ps);
  }

  /*
   * Ignore return value for now, but good programs should not party this way
   */

  madvise(addr, NR_PAGES * ps, MADV_MERGEABLE);
  getchar();
  return 0;
}
```

Recommended exercise

Compile and run the following program and try out the steps given below:

1. Run one instance of the program and see the output of the statistics specified above (run *cat /sys/kernel/mm/ksm/pages_** and observe the value of each of the parameters)
2. Run two and then three instances of the same program. Does the output change?
3. What happens if the *madvise(2)* hint is removed from the program?
4. What happens if we add a hint with MADV_UNMERGEABLE following the *getchar()* and add a new *getchar()* statement.

The other interesting utility in Fedora 12 is called *ksmtuned*. *ksmtuned* is a shell script that tries to automatically tune KSM based on the memory statistics that it reads from the system. It tracks committed memory (sum of all the memory being actually used by applications), the free memory (memory that can be freed easily and made available), and tunes the parameters we listed in the section above, to make KSM more effective. END

Acknowledgements

- The author would like to acknowledge Andrea Arcangeli, Mike Day and Amit Shah for their review of the article.

By: Balbir Singh

The author is a kernel hacker working with IBM's Linux Technology Centre. He has been around the IT industry for a long time and has used Linux in several domains. His interests include algorithms and computer simulation of architecture (MMIX). Balbir has been recognised by several awards and is also the maintainer of some Linux subsystems.